

The Battle of Architectures: Static Instruction Accumulation versus Dynamic Semantic Routing in Enterprise LLM Extraction

Tauan Vinicius Guahyba Sloboda

Abstract

Enterprise LLM systems routinely accumulate safety policies, compliance rules, formatting instructions, domain guidance, tone requirements, and exception handling inside long, static System Prompts. This practice is widespread, yet its real effect on token consumption, monetary cost, latency, operational stability, and extraction accuracy has remained largely unmeasured.

We conducted Experiment Two—*The Battle of Architectures*—as a controlled comparison between static instruction accumulation and dynamic semantic routing. Route A executed a single `gpt-4o-2024-11-20` call preceded by a synthetic 8,192-token corporate System Prompt. Route B used a two-stage architecture: `gpt-4o-mini-2024-07-18` converted a noisy user request into a compact Narrative Gravity formula, and `gpt-4o-2024-11-20` executed that formula against the same chaotic email thread. Both final calls used temperature 0, a 128-token output ceiling, identical source material, and no Structured Outputs.

Across 50 preregistered paired observations, Route B reduced total token consumption by **74.47%**, from 10,282.28 to 2,624.84 tokens per pipeline. It reduced observed monetary cost by **67.54%**, from US\$0.0136206 to US\$0.0044209 per extraction, despite paying for an additional model call. Under a no-cache counterfactual, the reduction increased to **78.87%**. Route B achieved **100% normalized material accuracy**, compared with **72%** for Route A. Route A failed the normalized extraction contract in 14 of 50 calls; Route B failed in none.

The latency result separated model service time from operational wall time. Route A completed its single inference faster on average—2.257 seconds versus 3.246 seconds for Route B's two sequential calls. Under token-per-minute admission control, however, Route A required an average operational wall time of 24.202 seconds, compared with 3.284 seconds for Route B. Route B therefore reduced operational wall time by **86.43%**.

These results establish dynamic semantic routing as the superior architecture for this enterprise extraction benchmark. It simultaneously reduced cost, reduced total token mass, eliminated normalized material errors, and prevented the rate-limit collapse produced by the hypertrophied static System Prompt.

1. Introduction

The standard enterprise response to LLM unpredictability is instruction accumulation. A deployment begins with a small System Prompt. Safety language is added. Then privacy rules, legal qualifications, audit requirements, tone guidance, formatting conventions, fallback logic, regional policies, product-specific exceptions, and instructions governing how the previous instructions should be reconciled. The prompt becomes an organizational sediment layer: every incident deposits another rule, while almost nothing is removed.

Prompt caching documentation itself identifies repeated System Prompts and common instructions as central reusable prefixes in production workloads. The infrastructure therefore recognizes static instruction accumulation as a normal operating pattern. Caching reduces the price and latency of exact-prefix reuse, but it does not remove those instructions from the model's active context.

This architecture is rarely measured as an architecture. Teams track whether the model follows policy, but not how much computational friction the accumulated policy creates. A static System Prompt is treated as fixed overhead rather than as an active component of inference. The result is a blind spot: organizations optimize model selection, quantization, caching, batching, and output length while leaving the semantic topology of the request unexamined.

The Semantic Thermodynamics framework treats that topology as an engineering variable. Its first article established Narrative Gravity as a structured constraint field composed of operational identity, objective, scope, negative constraints, output form, and failure state. Its second article formalized Narrative Pruning: the contraction of the effective output phase space before the executor begins materializing a response.

Within this framework, instruction volume and useful constraint are not equivalent. A relevant boundary removes degrees of freedom. A redundant boundary consumes context without reducing uncertainty. A conflicting boundary creates Structural Friction. A generic corporate rule that does not resolve any uncertainty in the current task remains semantically active while contributing no task-specific compression.

Transformers connect tokens through attention-based representations, making context an active computational field rather than a passive document attached to the request. Long-context research has also established that additional context does not translate automatically into robust use of information; performance changes with the position and distribution of relevant evidence. A larger context window provides capacity, not immunity from contextual interference.

Experiment Two asks a direct architectural question:

Can a two-stage system—a small semantic micro-router followed by a large executor—outperform a single large-model call governed by an 8K-token static System Prompt?

The answer is yes.

Route B reduced total tokens by 74.47%, reduced actual cost by 67.54%, reduced no-cache cost by 78.87%, and increased normalized material accuracy from 72% to 100%. Route A retained an advantage in mean service latency because it used one model call rather than two. That advantage disappeared at the operational layer, where its atomic token footprint forced prolonged admission waits and had already destroyed the first ungated execution of the benchmark.

The result is not a conventional cost-quality trade-off. Route B was both cheaper and more accurate.

2. Semantic Thermodynamics as an Architectural Model

2.1 Static instruction accumulation

Let:

- **S** denote the static corporate System Prompt;
- **U** denote the user request;
- **D** denote the source document;
- **M** denote the executor model.

Route A is defined as:

$$Y_a = M(S \oplus U \oplus D)$$

where $\oplus$ denotes contextual concatenation.

This architecture submits the complete policy layer on every request. Prompt caching changes how the repeated prefix is priced and prefetched, but the model still conditions its output on the full instruction field.

The static prompt therefore imposes two distinct costs:

$$C_a = C_{\text{prefill}}(S, U, D) + C_{\text{decode}}(Y_a)$$

and

$$F_a = F_{\text{relevance}} + F_{\text{redundancy}} + F_{\text{conflict}}$$

where F_a represents Structural Friction generated by instructions whose relevance, overlap, or priority must be resolved inside the executor.

2.2 Dynamic semantic routing

Route B inserts a semantic compilation stage. A small router $\mathbf{R}$ receives the noisy user request and generates a task-specific formula:

$$\mathbf{G} = \mathbf{R}(\mathbf{U})$$

with:

$$\mathbf{G} = (\Psi, \Omega, \phi, \bar{\mathbf{X}}, \mathbf{M}_o)$$

where:

- Ψ is the operational persona;
- Ω is the singular objective;
- ϕ is the evidence boundary;
- $\bar{\mathbf{X}}$ is the set of negative constraints;
- $\mathbf{M}_o$ is the output matrix.

The executor then receives only the compact formula and the source:

$$\mathbf{Y}_\beta = \mathbf{M}(\mathbf{G} \oplus \mathbf{D})$$

The total cost is:

$$\mathbf{C}_\beta = \mathbf{C}_r(\mathbf{U}) + \mathbf{C}_\square(\mathbf{G}, \mathbf{D})$$

Route B pays for an additional inference. That additional inference is not administrative overhead; it is semantic compilation. The router converts an open, noisy request into an execution field with a sharply reduced number of legitimate destinations.

2.3 Narrative Pruning

Let $\Gamma(\mathbf{P}, \mathbf{D})$ represent the effective set of output trajectories compatible with prompt $\mathbf{P}$ and document $\mathbf{D}$.

The static architecture exposes the executor to:

$$\Gamma_a = \Gamma(\mathbf{S} \oplus \mathbf{U}, \mathbf{D})$$

The dynamic architecture exposes it to:

$$\Gamma_\beta = \Gamma(\mathbf{G}, \mathbf{D})$$

Narrative Pruning constructs $\mathbf{G}$ so that irrelevant policy branches, discarded temporal versions, financial distractors, conversational padding, and non-terminal interpretations are excluded before executor decoding.

The objective is not minimal text. It is minimal **active semantic freedom**.

A long instruction can be efficient when every component resolves real ambiguity. A short instruction can be inefficient when it leaves the model to infer the task, scope, evidence

hierarchy, and output contract. The engineering target is the smallest coherent constraint field that preserves the complete material objective.

3. Experimental Design

3.1 Benchmark task

The benchmark used a seven-message corporate email thread involving three participants:

- Helena Duarte, an irritated client;
- Caio Mendes, a desperate and verbose junior employee;
- Ricardo Azevedo, an absent manager.

The thread contained deliberately engineered distractors:

- an original contract value of **R\$48,000.00**;
- an unapproved discount of **R\$6,000.00**;
- the amount actually payable, **R\$2,400.00**;
- an original date of **July 31, 2026**;
- email timestamps dated **August 10, 2026**;
- the client's final deadline of **August 12, 2026**;
- an irrelevant personal story involving a car, a sick cat, a tow truck, a phone battery, and VPN access;
- a causal admission establishing that Ricardo ordered the operational hold.

The required output was:

```
{"culprit":"Ricardo","amount":2400,"deadline":"2026-08-12"}
```

The gold standard was stored separately, hashed before execution, and never inserted into any model request.

3.2 Route A: static System Prompt

Route A used one call to:

gpt-4o-2024-11-20

Its system message contained a deterministic synthetic corporate System Prompt of exactly **8,192 local tokens**. Internally designated the “God Prompt” during implementation, the artifact simulated the accumulation of generic enterprise rules governing safety, privacy, legal caution, financial analysis, tone, completeness, auditability, formatting, ambiguity, and final inspection.

The System Prompt was synthetic. It did not reproduce, estimate, or claim access to any company's proprietary production prompt.

The user message contained:

1. the unchanged chaotic user request;
2. the unchanged email thread.

The API reported **10,242 prompt tokens** for every measured Route A call.

3.3 Route B: dynamic semantic routing

Route B used two sequential calls.

The micro-router used:

gpt-4o-mini-2024-07-18

It received only the chaotic user request and returned a strict internal object containing:

- persona;
- objective;
- scope;
- negative constraints;
- output matrix.

The router never received the email thread and therefore could not recover or leak the answer.

The compact router formula became the sole System Prompt for the executor:

gpt-4o-2024-11-20

The email thread became the executor's sole user message. The executor did not receive the original user request, the static System Prompt, or the gold standard.

GPT-4o and GPT-4o mini were selected through fixed snapshots. The published pricing used in the experiment was US\$2.50 per million uncached input tokens, US\$1.25 per million cached input tokens, and US\$10.00 per million output tokens for GPT-4o; and US\$0.15, US\$0.075, and US\$0.60 respectively for GPT-4o mini.

3.4 Controlled parameters

Both final executor calls used the same controlled configuration.

Executor snapshot: gpt-4o-2024-11-20

Temperature: 0

Top-p: 1

Maximum final tokens: 128

Streaming: Disabled

Tools: None

Final-call Structured Outputs: Disabled

Final-call JSON mode: Disabled

Structured Outputs were deliberately disabled in the final calls because native JSON adherence was an experimental outcome. Enforcing a schema at the API level would have converted the metric into a guaranteed property.

Only the internal micro-router used Structured Outputs. This ensured that the routing artifact remained machine-readable without artificially improving either final executor.

3.5 Stopping rule and paired design

The experiment preregistered exactly 50 logical A/B pairs. Temporary infrastructure failures did not consume an observation. A successful model response—correct, incorrect, malformed, refused, or truncated—was permanent experimental data and was never retried to improve quality.

Two unmeasured warm-up pairs preceded the main run. The 50 measured pairs alternated execution order:

- odd iterations: Route A, then Route B;
- even iterations: Route B, then Route A.

The run persisted:

- complete final outputs;
- per-call token telemetry;
- cached-token telemetry;
- model snapshots;
- request identifiers;
- service latency;
- admission waits;
- retry waits;
- monetary cost;
- output hashes;
- artifact hashes.

The analysis used 20,000 deterministic paired bootstrap resamples and exact two-sided sign tests excluding ties.

3.6 Two latency definitions

The experiment separated service latency from operational wall time.

For Route A:

$L_a(\text{service}) = L_{\text{executor}}$

For Route B:

$L_b(\text{service}) = L_{\text{router}} + L_{\text{executor}}$

Operational wall time included infrastructure admission:

$$L(\text{wall}) = L(\text{service}) + W_{\text{admission}} + W_{\text{retry}} + L_{\text{rejected}}$$

This distinction prevents a fast accepted request from being confused with an architecture that must wait tens of seconds before the API has enough token capacity to admit it.

3.7 The first execution collapsed under token pressure

The first version of Experiment Two launched Route A and Route B concurrently without rate-aware admission control.

It attempted 150 calls:

- 50 Route A calls;
- 50 Route B router calls;
- 50 Route B executor calls.

The run produced **69 HTTP 429 failures caused by tokens-per-minute limits**.

Route A: 3 successful, 47 failed.

Route B micro-router: 50 successful, 0 failed.

Route B executor: 28 successful, 22 failed.

Only **2 of 50 pairs** completed on both sides.

This was an architectural failure under resource contention. Route A injected 10,254 prompt tokens in a single atomic request. Its oversized contextual mass consumed the shared token envelope in large blocks. The Route B micro-router, operating on a compact request, completed all 50 calls. The Route B executor inherited part of the congestion on the shared GPT-4o budget but still completed more than nine times as many final calls as Route A.

The rerun introduced a rate-limit governor that read token-capacity headers, delayed calls before exhaustion, honored reset intervals, and executed the heavy GPT-4o requests sequentially. OpenAI exposes request and token limits, remaining capacity, reset intervals, project-token limits, and **Retry-After** for temporary 429 responses; the experiment used those controls as an explicit admission layer.

The governor did not make Route A efficient. It prevented Route A from collapsing.

4. Results

4.1 Run integrity

The governed English run completed:

- **50/50 measured pairs;**

- 100 final route observations;
- 150 measured logical model calls;
- 6 successful warm-up calls;
- 0 HTTP 429 responses;
- 0 retries;
- 36/36 validated publication hashes.

The API returned the requested snapshots throughout:

- executor: `gpt-4o-2024-11-20`;
- router: `gpt-4o-mini-2024-07-18`.

The complete run lasted **1,386.686 seconds**, or **23 minutes and 6.686 seconds**.

The append-only raw-attempt journal contained 154 of the 156 successful warm-up and measured call records. Two Route B executor entries—iterations 9 and 35—were absent from that journal. Their request IDs, outputs, token telemetry, costs, latencies, and hashes remained preserved and cross-validated in the primary CSV and final-output journal. No missing headers were reconstructed. The 100 final observations and all 650 recomputed paired deltas remained complete.

4.2 Main comparison

The central comparison produced the following results.

Mean prompt tokens

Route A: **10,242.00**

Route B: **2,500.84**

B – A: **–7,741.16**

Change versus A: **–75.58%**

Mean cached tokens

Route A: **9,909.76**

Route B: **860.16**

B – A: **–9,049.60**

Change versus A: **–91.32%**

Cache ratio

Route A: **96.756%**

Route B: **34.395%**

Difference: **–62.361 percentage points**

Mean uncached prompt tokens

Route A: **332.24**

Route B: **1,640.68**

B – A: **+1,308.44**
Change versus A: **+393.82%**

Mean completion tokens

Route A: **40.28**
Route B: **124.00**
B – A: **+83.72**
Change versus A: **+207.85%**

Mean total tokens

Route A: **10,282.28**
Route B: **2,624.84**
B – A: **-7,657.44**
Change versus A: **-74.47%**

Mean service latency

Route A: **2.257 s**
Route B: **3.246 s**
B – A: **+0.990 s**
Change versus A: **+43.86%**

Median service latency

Route A: **1.311 s**
Route B: **2.665 s**
B – A: **+1.354 s**
Change versus A: **+103.26%**

Service-latency p95

Route A: **6.383 s**
Route B: **4.359 s**
B – A: **-2.024 s**
Change versus A: **-31.71%**

Mean operational wall time

Route A: **24.202 s**
Route B: **3.284 s**
B – A: **-20.917 s**
Change versus A: **-86.43%**

Mean observed cost

Route A: **US\$0.0136206**
Route B: **US\$0.0044209**

B – A: **-US\$0.0091997**
Change versus A: **-67.54%**

Mean no-cache cost

Route A: **US\$0.0260078**
Route B: **US\$0.0054961**
B – A: **-US\$0.0205117**
Change versus A: **-78.87%**

Exact accuracy

Route A: **0/50**
Route B: **3/50**
Difference: **+3 outputs, or +6 percentage points**

Normalized material accuracy

Route A: **36/50**
Route B: **50/50**
Difference: **+14 outputs, or +28 percentage points**

Native JSON adherence

Route A: **0/50**
Route B: **3/50**
Difference: **+3 outputs, or +6 percentage points**

Primary contamination

Route A: **0/50**
Route B: **0/50**
Result: **Tie**

Courtesy tokens

Route A: **0**
Route B: **0**
Result: **Tie**

4.3 Token consumption

Route B used **2,624.84 total tokens per pipeline**, compared with **10,282.28** for Route A.

The reduction was:

$$1 - (2,624.84 / 10,282.28) = 74.47\%$$

Route B won total-token consumption in **50 of 50 paired observations**. The mean paired reduction was 7,657.44 tokens, with a 95% bootstrap interval of:

[-7,659.76, -7,654.76]

The two architectures distributed their tokens differently.

Route A used 10,242 prompt tokens and 40.28 completion tokens. Route B used 2,500.84 prompt tokens and 124 completion tokens. Route B's total completion count included the semantic compilation stage.

Route B micro-router

Prompt tokens: **490.00**

Completion tokens: **89.84**

Total tokens: **579.84**

Route B executor

Prompt tokens: **2,010.84**

Completion tokens: **34.16**

Total tokens: **2,045.00**

Complete Route B pipeline

Prompt tokens: **2,500.84**

Completion tokens: **124.00**

Total tokens: **2,624.84**

Aggregate completion tokens were higher for Route B because the router had to emit the Narrative Gravity formula. At the final executor stage, however, Route B used **34.16 completion tokens**, compared with **40.28** for Route A—a **15.19% contraction in final decoding**.

The router's output was therefore not free, but it purchased a cleaner executor state.

4.4 Monetary cost

Route A cost an average of:

US\$0.0136206

per extraction.

Route B cost:

US\$0.004420904

The observed reduction was:

67.54%

Across 50 observations:

Observed pricing

Route A mean: **US\$0.0136206**

Route B mean: **US\$0.0044209**

Route B saving: **67.54%**

Route A total: **US\$0.6810300**

Route B total: **US\$0.2210452**

Absolute Route B saving: **US\$0.4599848**

Counterfactual without prompt caching

Route A mean: **US\$0.0260078**

Route B mean: **US\$0.0054961**

Route B saving: **78.87%**

Route A total: **US\$1.3003900**

Route B total: **US\$0.2748052**

Absolute Route B saving: **US\$1.0255848**

Route A was **3.08 times more expensive** under observed caching and **4.73 times more expensive** under the no-cache counterfactual.

Route B won observed cost and no-cache cost in **50 of 50 pairs**.

The micro-router cost:

US\$0.000127404

per pipeline, representing less than 3% of Route B's total cost. Its additional inference was fully compensated by the executor context it removed.

4.5 Prompt caching

Route A achieved a **96.756% cache ratio**:

- 9,909.76 cached tokens;
- 332.24 uncached tokens.

Route B achieved a **34.395% cache ratio**:

- router: 0%;
- executor: 42.776%;
- complete route: 34.395%.

Prompt caching automatically reuses eligible repeated prefixes and charges cached GPT-4o input at a discounted rate. It therefore gave Route A its strongest available economic defense.

That defense was insufficient.

Even with almost the entire static prefix cached, Route A remained 67.54% more expensive than Route B. Removing caching widened Route B's advantage to 78.87%. The financial conclusion remained unchanged in both regimes.

4.6 Service latency

Route A won mean service latency:

2.257 s vs. 3.246 s

This is the direct cost of one hop versus two. Route B first paid 2.249 seconds for semantic compilation and then 0.998 seconds for executor inference.

Micro-router

Mean: **2.249 s**

Median: **1.778 s**

p95: **2.752 s**

Executor

Mean: **0.998 s**

Median: **0.816 s**

p95: **2.031 s**

Complete Route B pipeline

Mean: **3.246 s**

Median: **2.665 s**

p95: **4.359 s**

Route A was faster in 37 of 50 service-latency pairs. Route B was faster in 13.

The paired mean difference was:

$L_B(\text{service}) - L_A(\text{service}) = 989.676 \text{ ms}$

with a 95% bootstrap interval of:

[105.843, 2,075.945] ms

Route B nevertheless achieved the lower p95:

4.359 s vs. 6.383 s

Route A won average and median service latency; Route B produced the tighter p95 tail. Route B's largest service outlier reached 22.335 seconds, while Route A's largest reached 8.685 seconds.

4.7 Operational wall time and admission pressure

Operational wall time reversed the service-latency result:

24.202 s for Route A vs. 3.284 s for Route B

Route B reduced mean operational wall time by:

86.43%

The governor imposed **1,096.101 seconds** of admission waiting across the experiment, all on Route A. Twenty-four of 50 Route A observations required admission waits, with approximately 45.67 seconds of waiting among those affected calls. Route B required none.

Route A

Mean wall time: **24.202 s**

Median wall time: **7.257 s**

p95 wall time: **51.755 s**

Total admission wait: **1,096.101 s**

HTTP 429 responses: **0**

Retries: **0**

Route B

Mean wall time: **3.284 s**

Median wall time: **2.714 s**

p95 wall time: **4.399 s**

Total admission wait: **0 s**

HTTP 429 responses: **0**

Retries: **0**

The paired mean wall-time delta was:

-20.917 s

with a 95% bootstrap interval of:

[-27.300, -14.624] s

Route B won 28 wall-time pairs and Route A won 22. The difference between the modest win count and the large mean reduction arose from magnitude: Route A's admission penalties approached 50 seconds when they occurred.

The static architecture therefore had two latency identities:

1. a faster accepted inference;
2. a far slower admissible production operation under the project's token envelope.

4.8 Extraction accuracy

The strict exact-accuracy metric required:

- native JSON;
- no Markdown;
- exactly three keys;
- exact types;
- "Ricardo";
- integer 2400;
- ISO date "2026-08-12".

Under that standard:

Route A

Exact accuracy: **0/50 = 0%**

Normalized material accuracy: **36/50 = 72%**

Clean native JSON: **0/50 = 0%**

Markdown present: **50/50 = 100%**

Refusals: **0/50**

Truncations: **0/50**

Route B

Exact accuracy: **3/50 = 6%**

Normalized material accuracy: **50/50 = 100%**

Clean native JSON: **3/50 = 6%**

Markdown present: **47/50 = 94%**

Refusals: **0/50**

Truncations: **0/50**

Route A produced two output classes.

Thirty-six responses contained the correct underlying values but expanded the culprit to the full name and wrapped the object in Markdown.

The JSON content was:

```
{
  "culprit": "Ricardo Azevedo",
  "amount": 2400.00,
  "deadline": "2026-08-12"
}
```

Fourteen responses also renamed the required field.

The JSON content was:

```
{
  "culprit": "Ricardo Azevedo",
  "amount_due": 2400.00,
  "deadline": "2026-08-12"
}
```



```
}
```

Those 14 responses failed the normalized material contract because the requested **amount** field was absent. In an integration pipeline, the correct number under the wrong key is a failed extraction.

Route B produced 47 materially correct responses inside Markdown fences.

The JSON content was:

```
{
  "culprit": "Ricardo",
  "amount": 2400,
  "deadline": "2026-08-12"
}
```

Three Route B responses satisfied the complete native contract:

```
{"culprit":"Ricardo","amount":2400,"deadline":"2026-08-12"}
```

Route A failed normalized material extraction in **14 of 50 observations**. Route B failed in **0 of 50**.

4.9 JSON adherence

Neither architecture achieved acceptable native JSON reliability through prompting alone.

Route A produced clean JSON in 0% of calls. Route B produced clean JSON in 6%. The dominant failure was not semantic confusion but Markdown serialization.

This result separates two control layers:

- Semantic Thermodynamics controlled **what** the model extracted.
- Free-form decoding remained unreliable for **how** the result was serialized.

Structured Outputs solves the second layer by constraining responses to a supplied JSON Schema. The next architecture therefore couples Narrative Pruning with syntactic decoding constraints rather than asking semantic instructions to perform both jobs.

4.10 Router behavior

The micro-router generated formulas containing between 85 and 110 tokens.

Mean formula size: 89.84 tokens

Median: 89 tokens

Minimum: 85 tokens

Maximum: 110 tokens

Unique exact formulas: 24

Most common formula: 6/50

The router was not lexically deterministic. It was operationally deterministic.

Across 24 distinct formulations, it consistently preserved the same execution invariants:

- contract-analysis persona;
- identification of the causal actor;
- extraction of the currently payable amount;
- extraction of the final client deadline;
- rejection of superseded values, old dates, personal stories, and provisional statements;
- exact three-field output matrix.

Lexical variation did not create semantic drift. The executor achieved 50/50 normalized material accuracy.

4.11 Contamination and syntactic friction

Both architectures completely suppressed the preregistered distractors.

cat

Route A: 0

Route B: 0

car

Route A: 0

Route B: 0

battery

Route A: 0

Route B: 0

48,000

Route A: 0

Route B: 0

6,000

Route A: 0

Route B: 0

Neither route repeated the irrelevant personal story, original contract value, or rejected discount.

Both routes also produced zero detected courtesy wrappers:

- no “Sure”;

- no “Here is”;
- no “Based on the email thread”;
- no “Hope this helps.”

Semantic contamination and conversational friction were ties.

4.12 Paired statistical results

The paired results were as follows.

Prompt tokens

Mean B – A: **-7,741.16**

Bootstrap 95% CI: **[-7,742.32, -7,739.82]**

B wins: **50**

A wins: **0**

Ties: **0**

Total tokens

Mean B – A: **-7,657.44**

Bootstrap 95% CI: **[-7,659.76, -7,654.76]**

B wins: **50**

A wins: **0**

Ties: **0**

Completion tokens

Mean B – A: **+83.72**

Bootstrap 95% CI: **[82.38, 85.20]**

B wins: **0**

A wins: **50**

Ties: **0**

Service latency

Mean B – A: **+989.68 ms**

Bootstrap 95% CI: **[105.84, 2,075.94]**

B wins: **13**

A wins: **37**

Ties: **0**

Operational wall time

Mean B – A: **-20,917.31 ms**

Bootstrap 95% CI: **[-27,300.26, -14,624.33]**

B wins: **28**

A wins: **22**

Ties: **0**

Observed cost

Mean B – A: **-US\$0.0091997**

Bootstrap 95% CI: **[-0.0098112, -0.0087231]**

B wins: **50**

A wins: **0**

Ties: **0**

No-cache cost

Mean B – A: **-US\$0.0205117**

Bootstrap 95% CI: **[-0.0205218, -0.0205033]**

B wins: **50**

A wins: **0**

Ties: **0**

Exact accuracy

Mean B – A: **+0.06**

Bootstrap 95% CI: **[0.00, 0.14]**

B wins: **3**

A wins: **0**

Ties: **47**

JSON adherence

Mean B – A: **+0.06**

Bootstrap 95% CI: **[0.00, 0.14]**

B wins: **3**

A wins: **0**

Ties: **47**

The paired analysis establishes three complete dominances:

- Route B used fewer total tokens in every pair.
- Route B cost less under observed caching in every pair.
- Route B cost less without caching in every pair.

Route A's complete dominance was limited to aggregate completion-token count, which included Route B's router output, and to service-latency wins in 37 of 50 pairs.

5. Discussion

5.1 Caching discounts the static prompt; it does not neutralize it

The strongest defense of static instruction accumulation is prompt caching. Route A achieved a 96.756% cache ratio. In 49 of 50 pairs, it received more cached tokens than Route B. Its median uncached input was only 130 tokens.

Yet Route A remained **3.08 times more expensive**.

This result exposes the boundary of caching. Caching reduces the cost of repeated prefill work. It does not remove the cached instructions from the model's semantic field, does not reduce the number of instructions the executor must obey, and does not guarantee that those instructions converge on the current task. The cached System Prompt remained 8,192 tokens of active policy.

Even under this exceptional cache ratio, Route B was cheaper and more accurate. Under the no-cache calculation, its financial advantage widened from 67.54% to 78.87%.

The decisive result is therefore not input compression alone. Route B also changed executor behavior. Its executor used 34.16 completion tokens rather than Route A's 40.28, a 15.19% reduction, while eliminating all normalized material failures. The total Route B completion count was higher only because it included the router's 89.84-token semantic compilation.

The micro-router converted generic uncertainty into a clean field before the large model touched the source. It selected the persona, fixed the objective, bounded the evidence, removed superseded values, and crystallized the output contract. The executor entered decoding with the phase space already pruned.

The effect is measurable in final-stage tokens, material accuracy, and cost. Narrative Pruning operates in the physical execution of decoding.

5.2 Route B improved cost and quality simultaneously

Optimization systems often trade quality for efficiency. Smaller models reduce cost but lose accuracy. Shorter prompts save tokens but omit necessary context. Aggressive compression increases ambiguity.

Route B did not follow that pattern.

It reduced observed cost by 67.54% and raised normalized material accuracy from 72% to 100%. It reduced total tokens in all 50 pairs and eliminated all 14 normalized extraction failures produced by Route A.

The static System Prompt was not merely expensive. It was less precise.

Its generic formatting and completeness rules failed to preserve the exact first-name requirement in every output. In 14 cases, it also allowed the model to substitute `amount_due` for the required `amount` key. The prompt contained thousands of tokens about quality and structure, yet failed to enforce the small output contract that mattered.

Route B used approximately 90 router tokens to generate a task-specific contract and delivered the correct three material values in all 50 calls.

This is the central architectural result:

Dynamic semantic routing did not exchange quality for efficiency. It improved both.

5.3 Two hops lose service latency and win operational stability

Route B's service-latency disadvantage is structural. It performs two sequential model calls. Its micro-router alone averaged 2.249 seconds, more than Route A's median complete service time.

For latency-sensitive workloads operating far below token limits, Route A retains a real one-hop advantage. Its mean service latency was 989.68 ms lower.

The production picture changes under quota.

The first ungated run demonstrated that Route A could not sustain the experiment's original concurrent schedule. It completed only 3 of 50 calls and absorbed 47 TPM rejections. The rerun required a governor to reserve capacity before admitting its 10K-token requests.

That governor assigned 1,096.101 seconds of waiting to Route A and none to Route B. Route A's mean operational wall time rose to 24.202 seconds. Route B remained at 3.284 seconds.

The static architecture was faster only after the infrastructure had already waited long enough to make it safe.

This is why service latency and operational latency must remain separate metrics. Service latency describes the accepted model call. Operational wall time describes the deployable system under its actual resource envelope.

Route A won the first. Route B won the second by 86.43%.

5.4 The failed first run was an architectural measurement

The 69 HTTP 429 failures from the first run were not disposable setup noise. They exposed a resource property of the architecture.

A request carrying 10,254 prompt tokens consumes token capacity in a large atomic block. When repeated concurrently, it reaches the project TPM ceiling rapidly. The mini-router's compact request never failed. The B executor, despite sharing GPT-4o capacity, failed less than half as often as Route A.

The second run did not refute the first. It operationalized its lesson. Admission control transformed rejection into waiting, allowing all 50 pairs to complete. The burden remained visible as Route A wall time.

A production system cannot classify rate-limit collapse as external to architecture when the architecture determines request mass. Token-per-minute consumption is part of system design.

5.5 Semantic control and syntactic control are separate layers

Route B solved the semantic task perfectly under normalized evaluation but still wrapped 47 of 50 outputs in Markdown.

This failure defines the boundary of Narrative Gravity. Semantic constraints organize meaning, evidence, and terminal content. They do not provide a hard grammar at the decoder.

The complete production architecture therefore has two complementary layers:

Dynamic Semantic Routing + Structured Decoding

The router decides what the output means. A JSON Schema guarantees how the output is serialized.

Semantic instructions should not be burdened with enforcing byte-level structure when the API exposes a dedicated syntactic mechanism. Structured Outputs guarantees schema adherence and removes the need to repeatedly intensify formatting language inside the System Prompt.

The next experiment couples these layers and measures their joint cost, latency, and accuracy.

5.6 The micro-router as reusable middleware

The router consumed 490 prompt tokens, generated approximately 90 completion tokens, and cost US\$0.0001274 per route. It produced 24 distinct formulas while preserving the same semantic invariants across all 50 executions.

This establishes the micro-router as a reusable middleware component rather than a task-specific trick.

Its role is analogous to a compiler front end:

1. parse noisy human intent;
2. identify the material target;
3. define the evidence boundary;
4. eliminate irrelevant branches;
5. emit a minimal execution contract;
6. hand the contract to the expensive model.

A production router can select not only the prompt but also the executor model, reasoning level, retrieval scope, schema, context window, and fallback policy. Narrative Gravity becomes an active routing layer rather than a static prompt-writing convention.

6. Limitations

This experiment establishes results for one enterprise extraction benchmark, one chaotic email thread, one account's rate-limit environment, one warmed-cache regime, and two fixed model snapshots.

The 8,192-token System Prompt was a synthetic extreme designed to isolate static instruction accumulation. It does not represent a measured industry average or reproduce a specific company's proprietary prompt.

Operational wall time reflects the preregistered admission governor and the token limits observed on the executing project. Service-latency results remain the architecture-independent API timing comparison; wall-time results capture deployment behavior under this resource envelope.

The experiment measured API telemetry rather than direct GPU utilization, power draw, or joules. The physical quantities reported here are token usage, service time, admission time, and monetary cost.

Final calls intentionally omitted Structured Outputs. Native JSON adherence therefore measures free-form instruction following, not the full reliability available from schema-constrained decoding.

Two successful Route B executor calls were absent from the raw append-only attempt journal. Their outputs, request identifiers, tokens, costs, latencies, and hashes remained preserved and cross-validated in the primary artifacts. The final 100 observations and paired analysis were complete.

Programming, creative generation, mathematical reasoning, retrieval, tool use, and other domains lie outside the tested scope. The next experimental series extends the architecture across those task classes.

7. Conclusion

For this benchmark, dynamic semantic routing defeated static instruction accumulation in the metrics that determine enterprise viability: Route B reduced total tokens by 74.47%, reduced observed cost by 67.54%, reduced no-cache cost by 78.87%, raised normalized material accuracy from 72% to 100%, and reduced operational wall time by 86.43%. The Semantic Thermodynamics framework explains the result: the micro-router imposed Narrative Gravity before executor inference, reduced realization entropy, removed superseded and irrelevant branches, and pruned unproductive inference paths. Route A retained a one-hop advantage in mean service latency, but its 10K-token request footprint collapsed the first ungated experiment with 47 failures in 50 calls and later required more than 18 minutes of admission waiting to complete the governed run. That failure was not an incident outside the benchmark; it was the involuntary demonstration that hypertrophied static System Prompts are fragile under resource contention. The next architecture combines dynamic semantic routing with syntactic decoding constraints and extends the benchmark across additional domains.

References

1. Sloboda, T. V. G. (2026). *Semantic Thermodynamics: A Framework for LLM Entropy Minimization*. Semantic Thermodynamics Research Series.
2. Sloboda, T. V. G. (2026). *The Collapse of Phase Space: Narrative as an Operational Thermodynamic Quantity in Inference Pruning*. Semantic Thermodynamics Research Series.
3. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems, 30.
4. Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). *Lost in the Middle: How Language Models Use Long Contexts*. Transactions of the Association for Computational Linguistics, 12, 157–173.
5. OpenAI. *Prompt Caching*. OpenAI API Documentation.
6. OpenAI. *Rate Limits*. OpenAI API Documentation.
7. OpenAI. *GPT-4o Model: Capabilities, Pricing, and Snapshots*. OpenAI API Documentation.
8. OpenAI. *GPT-4o mini Model: Capabilities, Pricing, and Snapshots*. OpenAI API Documentation.
9. OpenAI. *Structured Model Outputs*. OpenAI API Documentation.